TI-2600 Laboratório de Desenvolvimento Multiplataforma

Antonio Carvalho - Treinamentos

KMP Compose Navigation

KMP Compose Navigation

- A navegação determina como o usuário pode se mover dentro da aplicação.
- Existem algumas bibliotecas de navegação para Kotlin Multiplataforma, como Voyager, Decompose, Appyx e Navigation3.
- Navigation3 é uma biblioteca para Kotlin totalmente feita em Compose, funcionando bem no Kotlin Multiplatforma.
- Uma das vantagens do uso desta biblioteca é o controle total do *backstack*.

KMP Compose Navigation

- A navegação no Kotlin Multiplataforma utiliza os seguintes conceitos:
- **Navigation Graph** - descreve todos os destinos possíveis dentro do aplicativo e as conexões entre eles. Grafos de navegação podem ser aninhados para acomodar subfluxos no seu app.
- **Destination** - é um nó no grafo de navegação que pode ser navegado. Quando o usuário navega até o destino, o aplicativo exibe seu conteúdo.
- **Route** - identifica um destino e define os argumentos necessários para navegar até ele, mas não faz nada para descrever a interface. Dessa forma, os dados ficam separados da representação, o que permite manter cada parte da implementação da interface independente da estrutura geral do app. Isso facilita, por exemplo, testar e reorganizar *composables* no seu projeto.
- **BackStack** – contém a pilha de navegação, guardando em quais telas foram visitadas pelo usuário. De forma que ao clicar no botão **back** é possível voltar a tela anterior, removendo-a da pilha.

KMP Navigation 3 – Como instalar

- Para instalar o Navigation 3 é preciso alterar o arquivo **libs.versions.toml**

```
[versions]
...
androidx-lifecycle = "2.10.0-alpha07"
cmp-navigation3 = "1.0.0-alpha06"
cmp-navigation3-browser = "0.3.1"
androidx-adaptive = "1.3.0-alpha03"

[libraries]
...
androidx-lifecycle-viewmodel-nav3 = {
      module = "org.jetbrains.androidx.lifecycle:lifecycle-viewmodel-navigation3",
      version.ref = "androidx-lifecycle" }
androidx-material3-adaptive-nav3 = {
      module = "org.jetbrains.compose.material3.adaptive:adaptive-navigation3",
      version.ref = "androidx-adaptive" }
androidx-nav3-ui = {
      module = "org.jetbrains.androidx.navigation3:navigation3-ui",
      version.ref = "cmp-navigation3" }
navigation3-browser = {
      module = "com.github.terrakok:navigation3-browser",
      version.ref = "cmp-navigation3-browser" }

[bundles]
...
navigation3 = [
    "androidx-lifecycle-viewmodel-nav3", "androidx-material3-adaptive-nav3", "androidx-nav3-ui" ]
```

KMP Navigation 3 – Como instalar

- Adicione as dependências no arquivo **build.gradle.kts** do módulo e troque a versão do compile SDK para deixar no mínimo a versão 36.

```
kotlin {
    android {
        ...
        compileSdk = 36
    ...
    sourceSets {
        commonMain.dependencies {
            ...
            implementation( libs.bundles.navigation3 )
        }
        wasmJsMain.dependencies {
            ...
            implementation( libs.navigation3.browser )
        }
}
```

KMP Navigation 3 – Criando as Rotas

- Os primeiros códigos a serem criados devem ser as rotas possíveis de navegação. Que podem ser criadas como classes ou interfaces, desde que herdem de **NavKey**

```
import androidx.navigation3.runtime.NavKey
import kotlinx.serialization.Serializable

@Serializable
sealed interface MainRoute : NavKey {
    @Serializable
   data object Categoria : MainRoute
    @Serializable
    data class Produto(val id : Int) : MainRoute
}
```

KMP Navigation 3 – Criando as Rotas

- Uma vez que as rotas estão criadas, precisa criar o **backstack**
- O **backstack** nada mais é do que um lista de elementos do tipo **NavKey**

```
import androidx.navigation3.runtime.NavKey
val backstack = remember { mutableStateListOf<NakKey>(MainRoute.Categoria) }
```

Ou

```
import androidx.navigation3.runtime.NavKey
val backstack = remember {
        mutableStateOf<NavBackStack<NavKey>>(NavBackStack(MainRoute.Categoria))     }
```

Ou

```
import androidx.navigation3.runtime.rememberNavBackStack
import androidx.navigation3.runtime.NavKey
import androidx.navigation3.runtime.samples.RememberNavBackStackSamples.Details
import androidx.navigation3.runtime.samples.RememberNavBackStackSamples.Home
import androidx.savedstate.serialization.SavedStateConfiguration

val config = SavedStateConfiguration {
       serializersModule = SerializersModule {
              polymorphic(baseClass = NavKey::class) {
                     subclass(serializer = MainRoute.Categoria.serializer())
                     subclass(serializer = MainRoute.Produto.serializer())
              }
      }
}

val backstack = rememberNavBackStack(config, MainRoute.Categoria)
```

KMP Navigation 3 – Criando as Rotas

- Defina agora onde a navegação irá exibir as telas, e quais telas serão exibidas.

```
import androidx.navigation3.runtime.entryProvider
import androidx.navigation3.ui.NavDisplay

NavDisplay (
    backStack = backstack,
    entryProvider = entryProvider<NavKey> {
        entry<MainRoute.Categoria> {
            Text(text = "Tela de Categoria",
                color = MaterialTheme.colorScheme.onBackground,
                fontSize = 16.sp)
        }
        entry<MainRoute.Produto> {
            Text(text = "Tela de Produto",
                color = MaterialTheme.colorScheme.onBackground,
                fontSize = 16.sp)
        }
    }
}
```

KMP Navigation 3 – Criando as Rotas

- A navegação consiste em **adicionar** e **remover rotas** no **backstack**. Sendo que a última rota adicionada é a que estará visível.

```
import androidx.navigation3.runtime.entryProvider
import androidx.navigation3.ui.NavDisplay
...
entry<MainRoute.Categoria> {
     Text(text = "Tela de Categoria",
          color = MaterialTheme.colorScheme.onBackground,
          fontSize = 16.sp)
     Button( onClick={
          backstack.removeLastOrNull()
          backstack.add( MainRoute.Produto )
     } ) {
          Text("Ir para Produtos")
     }
}
entry<MainRoute.Produto> {
     Text(text = "Tela de Produto",
          color = MaterialTheme.colorScheme.onBackground,
          fontSize = 16.sp)
     Button( onClick={ backstack.add( MainRoute.Categoria ) } ) {
          Text("Ir para Categorias")
     }
}
```